

CSCE 640 QUANTUM ALGORITHMS · MODULE 08 · LECTURE NOTES

Phase Estimation and Shor

The one dramatic result, assembled from parts you already have.

MODULE OUTCOMES

- Derive phase estimation from the QFT.
- Explain the reduction from factoring to order finding.
- Assemble Shor's algorithm and state its cost.
- Explain what it does and does not break.
- State the resource reality for a useful instance.

1 Phase estimation

given U and $|u\rangle$ with $U|u\rangle = e^{2\pi i\phi}|u\rangle$, estimate ϕ

- **The eigenvalue's phase is the thing you want, and it is not directly measurable** — a global phase is invisible (Module 02 §1) — **so it has to be moved into amplitudes of a separate register** before it can be read.
- **Apply controlled- U^{2^k} with control qubit k held in superposition: phase kickback (Module 03 §3)** writes $e^{2\pi i \cdot 2^k \phi}$ onto that control qubit, leaving the eigenvector untouched — which is why $|u\rangle$ being an eigenvector is the whole trick.
- **The control register now holds a phase pattern that is periodic in ϕ — so the inverse QFT turns it into a peak, and measuring gives ϕ to m bits with m control qubits** (Module 07 §1's period-becomes-peak, used in the direction that extracts the answer).
- **Phase estimation moves an eigenvalue's phase into a measurable register** — and **almost every exponential-advantage algorithm in the subject is an instance of it**, which is why it rather than Shor is the thing to learn. **Phase estimation is the general tool; Shor is one application of it.**

AND PERIOD FINDING IS PHASE ESTIMATION ON THE SHIFT OPERATOR

Define U as multiplication by a modulo N — the map $|x\rangle \rightarrow |ax \bmod N\rangle$ — and its eigenvalues have phases that are multiples of $1/r$, where r is the multiplicative order of a , which is exactly the period you want.

So estimating the phase estimates a multiple of $1/r$ — one random multiple per run — exactly as Simon's algorithm returned one random linear constraint per run (Module 05 §3), which is why Simon was worth studying first.

And r is recovered from that estimate by continued fractions — classical, cheap, and the step that makes the whole thing work with only a constant expected number of repetitions rather than many.

Which is why the chain QFT $\rightarrow$ phase estimation $\rightarrow$ period finding $\rightarrow$ factoring is worth seeing as a single thing rather than four topics — it is the subject's one route to a superpolynomial advantage, and everything else in Module 01 §3's list is either simulation or quadratic.

2 Factoring reduces to order finding

TO FACTOR N :

- 1 pick a random $a < N$
- 2 if $\gcd(a, N) \neq 1$, you are lucky: it is a factor. Return it.
- 3 find r , the multiplicative order of $a \bmod N$
(THIS is the quantum part, section 1)
- 4 if r is odd, or $a^{(r/2)} = -1 \bmod N$, restart
- 5 otherwise $\gcd(a^{(r/2)} - 1, N)$ is a nontrivial factor, with probability at least $1/2$

WHY STEP 5 WORKS

$a^r = 1 \bmod N$, so N divides $a^r - 1$
 $= (a^{(r/2)} - 1)(a^{(r/2)} + 1)$
 N divides the product but neither factor
(that is what step 4 ruled out), so N must share part of itself with each -- and gcd finds the shared part.

EVERYTHING EXCEPT STEP 3 IS CLASSICAL, and the reduction was known before quantum computing.

Everything except order finding is classical, and the reduction predates quantum computing — the quantum contribution is exactly one subroutine. Seeing how small the quantum part is clarifies what was actually contributed: not a new way of thinking about factoring, but an efficient method for a single number-theoretic subproblem that was already known to suffice. It also shows why a fault in step 3 is detectable: the classical steps verify the answer in a few multiplications.

3 What it breaks

Primitive	Status under a large fault-tolerant machine	Why
RSA	Broken.	Factoring, directly.
Diffie-Hellman and DSA	Broken.	Discrete logarithm, by the same phase-estimation machinery.
Elliptic curve cryptography	Broken.	Discrete logarithm in a different group, and with <i>smaller</i> resource requirements than RSA.
AES-256	Weakened to roughly 128-bit security.	Grover only (Module 06) — and that is entirely survivable by using larger keys.
SHA-256 and SHA-3	Weakened similarly.	Grover on preimages; collision resistance is affected less.
Lattice-based and hash-based signatures	No known quantum attack.	Which is precisely why they are the standardised replacements (CSCE 711 Module 13).

Symmetric cryptography survives with larger keys — the break is specific to the number-theoretic public-key primitives, and the replacements already exist and are standardised. The asymmetric/symmetric distinction is the one people most often lose, and 'quantum computers will break all encryption' is wrong in a way that matters: it suggests nothing can be done, when in fact the migration path is specified.

AND THE HARVEST-NOW-DECRYPT-LATER PROBLEM MAKES THIS URGENT DESPITE THE MACHINE NOT EXISTING

Encrypted traffic recorded today can be decrypted whenever such a machine appears — so anything that must stay secret for twenty years is already exposed, if it was sent under RSA or ECDH key exchange, and recording is cheap.

Which makes migration a present-tense engineering problem, independent of whether the machine arrives in ten years, in fifty, or never — and that independence is what makes the argument robust.

And the standards exist: lattice-based key encapsulation and both lattice- and hash-based signatures are standardised and deployable now (CSCE 711 Module 13's post-quantum section), with hybrid deployments already common.

So the honest position is: migrate for the recorded-traffic reason, not because the machine is imminent — and do not claim it is imminent in order to motivate the migration, which has been done and which damages the argument's credibility when the predicted date passes.

4 The resource reality

FACTORING A 2048-BIT RSA KEY NEEDS MILLIONS OF PHYSICAL QUBITS AND HOURS OF RUNTIME, ON A MACHINE NOBODY HAS

The careful published estimates are in the millions of physical qubits — a few thousand logical qubits multiplied by Module 10 §4's error-correction overhead — running for hours, with the architectural assumptions stated explicitly, which is what makes those papers worth reading.

Against current devices of hundreds of noisy physical qubits with no useful error correction — which is a gap of three to four orders of magnitude in qubit count, not a matter of waiting one more hardware generation.

And the estimates have been falling steadily as the algorithms, the codes, and the magic-state factories improve — which is genuine progress, and is still progress within those orders of magnitude rather than across them.

So quote the number rather than the possibility — which is this course's practice throughout and is Module 13's explicit requirement: a claim about capability needs a resource count attached, and a claim without one is not checkable.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Shor — Polynomial-time algorithms for prime factorization and discrete logarithms (free)	§§1 and 2 in the original — and it is readable, which is not true of every landmark paper.
Kitaev — Quantum measurements and the abelian stabilizer problem (free)	§1 — phase estimation in the framing that makes it the general tool rather than a step in Shor.
Gidney & Ekerå — How to factor 2048 bit RSA integers in 8 hours (free)	§4's numbers — the careful estimate, with every assumption stated. This is the paper to cite.
NIST post-quantum cryptography standards (free)	§3's replacements — the standardised algorithms and the migration guidance.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 State the phase estimation problem and its circuit.
- 2 Show where phase kickback enters, explicitly.

- 3 **Implement phase estimation** for a single-qubit U with known eigenvalue.
- 4 **Verify the precision** against the number of control qubits.
- 5 **Find the order of several $a \bmod N$** classically, and check the reduction's steps.
- 6 **Prove step 5** of the reduction in your own words.
- 7 **Run Shor's algorithm** on $N = 15$ and $N = 21$ in simulation.
- 8 **Classify ten cryptographic primitives** as broken, weakened, or unaffected.
- 9 **State the harvest-now argument** and its independence from the timeline.
- 10 **Read the 2048-bit estimate** and list its three strongest assumptions.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the phase estimation problem and the three circuit steps.
- 2 Why must the input be an eigenvector?
- 3 How is period finding an instance of it?
- 4 How is r recovered, and how many runs are needed?
- 5 Give the factoring reduction, and say which step is quantum.
- 6 Why does the gcd step produce a factor?
- 7 Name three broken primitives and three that survive.
- 8 What is the distinction people most often lose?
- 9 State the harvest-now-decrypt-later argument.
- 10 Give the resource estimate and the current gap.